

Локализация C++

Для локализации строковых констант в исходном коде C++ необходимо использовать функцию `rk()`. Она принимает следующие параметры:

1. Строка для перевода. Строка, которую механизм интернационализации будет искать в словаре.
2. Контекст. Строка, описывающая [контекст перевода](#) для переводчика. Является необязательным параметром.
3. Число. Число, которое задает количество объектов для выбора склонения.



Импортирование функции `rk()`

Для работы функции `rk()` необходимо явно импортировать библиотеку `#include <sbis-lib/utils/rk.hpp>`.

Локализация строк

```
1 #include <sbis-lib/utils/rk.hpp>
2 ...
3 LogMsg( rk( L"функция checkout устарела"_sv, L"Мой функционал"_sv ) );
```

В качестве аргументов функции не могут выступать переменные. Используйте только константы. Допустимо писать в аргументах только неименованные строковые константы.

```
1 const String BLABLA = L"foo bar"_sv;
2
3 // Неправильно! Genie не сможет в окне назначения переводов показать
  фразу foo bar.
4 rk( BLABLA );
5
6 // Правильно.
7 rk( L"foo bar"_sv );
```

При использовании в тексте исходного кода кавычек, их требуется экранировать.

```
1 rk( L"000 \"Тензор\""_sv );
```

Конкатенация строк

Склейка локализуемых строк запрещена.

```
1 // Неправильно
2
3 Error<Exception>( rk( L"Ошибка"_sv ) + ': ' + message + ' '
4                  + rk( L"с кодом"_sv ) + ' ' + code );
```

Для подстановки динамических данных в локализуемые строки необходимо использовать плейсхолдеры и функцию [sbis::Format](#):

```
1 Error<Exception>( Format( rk( L"Ошибка: \"%1%\" с кодом \"%2%\""_sv ),
  message, code ) );
```

Нумерация плейсхолдеров начинается с 1. Переменные перечисляются друг за другом через "%".

Подстановка выполняется **после** вызова метода `rk`, это значит что переводы на все языки должны содержать плейсхолдеры для форматирования.

Пробелы, управляющие символы в начале и в конце локализуемых строк

В переводах запрещено использование пробелов и [управляющих символов](#) (`\t` — горизонтальная табуляция; `\n` — перевод строки; `\r` — возврат каретки) в начале и в конце строки:

1. Локализуемая строка не должна содержать символы пробелов в начале или в конце:

```
1 // Неправильно
2 rk( L" Текст "_sv)
3
4 // Правильно
5 rk( L"Текст"_sv)
```

2. Локализуемая строка не должна содержать символы перевода строки.

```
1 // Неправильно
2 rk( L"\nТекст"_sv )
3
4 // Правильно
5 L"\n"_sv + rk( L"Текст"_sv )
```

Ключи с невидимыми символами на концах строки сильно затруднят перевод. Например, строки Сохранить, \nСохранить и Сохранить придётся переводить отдельно.

Знаки препинания

Знаки препинания включают в локализуемую строку:

```
1 // Неправильно
2 rk( L"Ошибка"_sv ) + L"!"_sv
3 rk( L"Предупреждение"_sv ) + L": "_sv
4 rk( L"Удалить"_sv ) + L"? "_sv
5
6 // Правильно
7 rk( L"Ошибка!"_sv )
8 rk( L"Предупреждение:"_sv )
9 rk( L"Удалить?"_sv )
```

Склонение слова в зависимости от числа

При склонении слова в зависимости от числа предполагается, что ключ должен содержать предполагаемые окончания:

```
1 запись(-и, -ей)
```

К исходной строке добавляются скобки с вариантами окончания слова. Такой вид строки нужен только для того, чтобы выделить её в [словаре переводов](#) среди остальных строк. Добавленные в скобках окончания никак не повлияют на перевод или форму слова при локализации. Если на сервисе отключена локализация, то строка будет выводиться в исходном виде.

Сигнатура функции `rk()`:

```
1 // ЛОКАЛИЗАЦИЯ + КОНТЕКСТ + СКЛОНЕНИЕ
2 // keyword – ключ для поиска перевода.
3 // context – контекст перевода.
4 // numeric – число, соответствующее склонению.
```

```

5 rk( keyword, context, numeric );
6
7 // ЛОКАЛИЗАЦИЯ + СКЛОНЕНИЕ
8 rk( keyword, numeric );

```

Примеры:

```

1 // ЛОКАЛИЗАЦИЯ + СКЛОНЕНИЕ
2
3 // Вернёт запись или record.
4 rk( L"запись(-и,-ей)"_sv, 1 );
5
6 // Вернёт записи или records.
7 rk( L"запись(-и,-ей)"_sv, 2 );
8
9 // Вернёт записей или records.
10 rk( L"запись(-и,-ей)"_sv, 5 );
11
12 // Вернёт записи или records.
13 rk( L"запись(-и,-ей)"_sv, 1.5 );
14
15 // ЛОКАЛИЗАЦИЯ + КОНТЕКСТ + СКЛОНЕНИЕ
16
17 // Вернёт день или day.
18 rk( L"день(-я,-ей)"_sv, L"контекст"_sv, 1 );
19
20 // Вернёт дня или days.
21 rk( L"день(-я,-ей)"_sv, L"контекст"_sv, 2 );
22
23 // Вернёт дней или days.
24 rk( L"день(-я,-ей)"_sv, L"контекст"_sv, 5 );
25
26 // Вернёт дня или days.
27 rk( L"день(-я,-ей)"_sv, L"контекст"_sv, 1.5 );

```

В словарь перевода добавляем такие строки:

```

1 /* ru-RU.json */
2 {
3     "plural#запись(-и,-ей)": "запись|записи|записей|записи",
4     "контекст@@plural#день(-я,-ей)": "день|дня|дней|дня"
5 }
6
7 /* en-US.json */
8 {
9     "plural#запись(-и,-ей)": "record|records",
10    "контекст@@plural#день(-я,-ей)": "day|days"
11 }

```

Условие для локализации исходного кода C++

Чтобы иметь возможность локализации исходных файлов на C++, нужно указать для каждого [сервисного модуля](#) путь до директории их хранения. Путь указывают в свойствах сервисного модуля в опции Sources, он должен быть сформирован относительно файла *.s3mod текущего модуля.

